LSTM — Part 3: Next Word Predictor

---

1. Intro: What Is a "Next Word Predictor"?

A **Next Word Predictor** is a deep learning model that learns patterns in **text sequences** and predicts the **most likely next word** after a given sequence.

Example:

Input → "I am going to"

Output → "school"

The model *learns* that in many contexts, "school" is a likely continuation after "I am going to".

This type of model powers:

- **Text auto-completion**
- **Chatbots**
- **Speech recognition post-processing**
- **Machine translation**
- **Predictive keyboards (like on phones)**

---

2. Why LSTM for This Task?

Traditional neural networks (ANNs) or CNNs don't handle **sequences** well.

But text is inherently **sequential** — each word's meaning depends on the previous ones.

1. LSTMs are great because they:
2. Remember **long-term dependencies**
3. Handle **varying-length inputs**
4. Avoid **vanishing gradient problems** that affect basic RNNs
5. They understand context across sentences or even paragraphs — crucial for language modeling.

---

3. What Is a Language Model?

Before predicting words, we need the concept of a **language model**.

A **language model** assigns a probability to a sequence of words:

$$P(w_1, w_2, ..., w_t)$$

It tries to maximize the probability of the next word given the previous ones:

$$P(w_t | w_1, w_2, ..., w_{t-1})$$

LSTM-based models *learn* this probability distribution through training on text.

---

4. The Strategy — How to Train the Model

The video explains a simple and effective strategy to build a **next-word predictor** using LSTM. Let's break it down:

Step 1: Collect Text Data

We need a large amount of text — for example, lines from books, articles, or song lyrics. This text will be used to teach the model about word patterns and context.

Step 2: Tokenize the Text

Tokenization = splitting text into **tokens (words)** and converting them into numbers.

Example:

Text: "I love deep learning"

Tokens: [1, 2, 3, 4]

This numerical representation allows the model to process text mathematically.

Step 3: Create Sequences

We create **input-output pairs** where the model learns to predict the next word.

Example:

Input: "I love"

Output: "deep"

Input: "I love deep"

Output: "learning"

So the model learns:

If I see "I love deep", the next word is likely "learning".

Step 4: Padding Sequences

Neural networks expect equal-length inputs, so we **pad** shorter sequences with zeros.

Example:

["I love deep"] → [0, 1, 2, 3]

This ensures all sequences have the same length.

Step 5: Define the Model Architecture

Now comes the **LSTM model design**.

---

5. Architecture of the LSTM Next Word Predictor

A typical architecture explained in the video:

```
Input Layer → Embedding Layer → LSTM Layer(s) → Dense (Softmax) Output
```

Let's understand each component:

Layer	Function	Explanation
Embedding Layer	Converts tokens to vectors	Turns words into dense numerical vectors (captures semantic meaning)
LSTM Layer	Learns sequence patterns	Remembers context and order in the sequence
Dense Layer	Fully connected NN layer	Processes LSTM's output
Softmax Output Layer	Predicts next word	Outputs probability distribution over all possible words

---

How It Works Internally:

1. The **embedding layer** maps each word ID to a vector (like "love" → [0.21, -0.33, 0.85, ...]).
2. The **LSTM layer** reads the sequence of word embeddings and keeps track of context.
3. The **dense layer** and **softmax** predict which word comes next — the one with the **highest probability**.

Example (Simple Architecture)

```
model = Sequential()
model.add(Embedding(vocab_size, 100, input_length=seq_length))
model.add(LSTM(150, return_sequences=False))
model.add(Dense(vocab_size, activation='softmax'))
```

- vocab_size: number of unique words in your dataset
- seq_length: number of words in input sequences
- Output → probability of each possible next word

6. Training the Model

Training steps:

1. **Input:** Sequences of words (e.g., "I love deep")
2. **Target:** Next word ("learning")
3. **Loss Function:** Categorical cross-entropy (since it's multi-class prediction)
4. **Optimizer:** Adam (commonly used for LSTM-based models)

The model gradually learns how words follow each other.

7. Using the Model for Prediction

Once trained, you can predict text like this:

Example Code:

```
def predict_next_word(model, tokenizer, text_seq, max_length):
  encoded = tokenizer.texts_to_sequences([text_seq])[0]
  padded = pad_sequences([encoded], maxlen=max_length, truncating='pre')
  y_pred = np.argmax(model.predict(padded), axis=-1)
  for word, index in tokenizer.word_index.items():
    if index == y_pred:
      return word
```

Input: "I love deep"

Output: "learning"

This process can be repeated iteratively to generate entire sentences.

8. How to Improve Model Performance

The video lists several great tips for improvement:

1. Use More Data

The more text your model sees, the better it understands context and grammar.

2. Increase Sequence Length

Give the model longer input sequences — helps it learn more context.

3. Use Pretrained Embeddings

Instead of random embeddings, use pretrained ones like **GloVe** or **Word2Vec** to start with meaningful word vectors.

4. Add More LSTM Layers

Stacking multiple LSTM layers allows the model to capture more complex patterns.

5. Use Dropout

Prevents overfitting by randomly turning off neurons during training.

6. Tune Hyperparameters

Experiment with:

- Learning rate
- Batch size
- Hidden layer size
- Number of epochs

7. Try Bidirectional LSTMs

These process text in **both directions** — forward and backward — giving richer context.

---

9. Visual Summary

Here's how the full pipeline looks conceptually:

```
Text Corpus
  ↓
Tokenization → Sequence Creation → Padding
  ↓
Embedding → LSTM → Dense → Softmax
  ↓
Predicted Next Word
```

---

10. Example: Real Prediction Flow

Input:

"Deep learning models are"

Output:

"powerful"

You can then append this and continue predicting:

"Deep learning models are powerful tools for..."

That's how modern **text generation systems** work — predicting one word at a time.

---

11. Why It's Important

This simple project demonstrates:

- **How sequential data works**
- **How LSTM learns temporal relationships**
- **How neural networks generate language-like text**

You can later extend this concept to:

- Text summarization
- Machine translation
- Chatbots
- Music/poem generation

Summary Table

Step	Concept	Explanation
1	Tokenization	Convert words → numbers
2	Sequence Creation	Prepare training pairs
3	Padding	Equalize sequence lengths
4	Embedding	Create dense word vectors
5	LSTM	Learn sequence patterns
6	Dense + Softmax	Predict next word
7	Prediction	Generate text
8	Optimization	Improve model via tuning

In short:

The **Next Word Predictor** is a practical way to understand how LSTMs remember context, process sequences, and generate language — bringing theory to life.
